PROJECT PROPOSAL

Financial Statement Simulator

Internship scope and phased build plan (rough draft for review)

Prepared by Jaebum (Albert) Chung | Prepared for Chak | Status: draft for discussion | June 2026

Internship window: 31 August 2026 to 26 February 2027

1. Context and alignment

The enterprise-grade bar for this project has been the intent from the outset. The work was deferred until last year because the available tooling was not yet good enough for the purpose. Prior intern contributions (Hong Xiao and Elton) established the core of the simulator: an internally consistent, no-circularity accounting engine and autoregressive driver modeling validated on synthetic data.

What the project does not yet have, and what this proposal scopes, are the pieces the system hinges on: an extraction layer that reads real filings (US GAAP and IFRS) into structured statements, per-standard encode and decode adapters that map those statements to and from a common state space, the state space itself, and a credible relationship layer that maps scenarios to the flow variables. The target is an MVP whose simulated statements an accountant would deem usable, not a research prototype.

The work divides into two parts. **Part I**, the MVP and the focus of this proposal, is a faithful simulator: it faithfully reads a real filing into a structured statement and produces next-period statements that carry the same line items, comply with the source standard, and move in the right direction under scenarios. Its driver layer is a simple state-space model, which is stochastic by construction, so Part I already runs simple stochastic simulations: for a given scenario it samples the driver noise and propagates it through the engine to a distribution of internally consistent statements. **Part II**, a later and conditional extension, keeps the same machinery but replaces the simple noise with a learned, conditional joint distribution of the drivers, the main modelling IP, giving better-calibrated and correlated distributions. In the phased plan below, Part I is Phases 0 through 3, and Part II is Phase 4.

2. Definition of success (MVP acceptance bar)

For Part I (the MVP), the bar is concrete, and the driver models are kept simple and reasoned rather than learned. Given a prior-period statement and a scenario, the system should produce simulated next-period statements that:

- an accountant cannot identify as obviously wrong;
- preserve the same field set as the input statement (exact line-item parity with the firm's prior statement);
- conform to the applicable standard, US GAAP or IFRS, matching the source filing; and
- respond in the correct direction to scenario changes (for example, faster GDP growth raises income; intensifying industry competition lowers income).

Simple stochastic output. For a fixed scenario the driver noise is sampled to give a distribution of these statements rather than a single point; the accounting engine keeps every sampled path internally consistent, and the directional test applies to how the distribution moves when the scenario changes.

Precision asymmetry. Extraction, reading a filing into a structured statement, is held to a near-zero per-field error rate (target on the order of 10^{-8}, reached through redundant tools), because a misread figure invalidates everything downstream. Encoding that statement into the state space and back is exact by construction, governed by perfect reconstruction rather than a probabilistic error rate. The simulation output, by contrast, needs only to be solid, robust, and directionally credible rather than precisely forecast.

Perfect reconstruction. A structured statement encoded into the state space and decoded back under its own standard must be identical to the original. This lossless round trip is the correctness test for the encode/decode pair, distinct from reading the filing correctly; the formulation and why it holds are in Section 3.1.

Pass threshold. A pass is a set of firms across a set of scenarios, reviewed by a qualified accountant, with no statement judged obviously wrong. The specific counts and reviewer are to be confirmed (see open questions); the point is that "done" has a testable definition.

3. System architecture

The simulation core is common to both standards; the standard-specific work sits at the boundaries, where extraction reads a filing and the encode/decode adapters map it to and from the common state space. Five components, most of them net-new; the accounting engine is reused.

Filing (US GAAP or IFRS) → [Extract] → **structured statement** → [Encode] → **common state space** → [Driver-to-flow layer] (+ scenario) → [Accounting engine] → **common state space** → [Decode] → **next-period statement** (same standard, exact line items)

Figure 1. End-to-end pipeline: a filing is extracted into a structured statement, encoded into the common state space, advanced by the driver-to-flow layer and the accounting engine under a scenario, and decoded back to a next-period statement in the same standard and line items. Extraction (reading the filing) and encoding (re-representing it) are distinct stages with different error models. Colored bracketed terms are the processing components of Table 1; bold terms are the data representations that flow between them.

Component	Status	Role
Statement extractor	**Net-new**	Reads a filing (tagged data or PDF) into a structured statement in the source standard, using redundant cross-checking tools; carries the near-zero read-error target and is deterministic at run time (Section 5).
Encode/decode adapters (US GAAP, IFRS)	**Net-new**	Losslessly map a structured statement to and from the common state space, one pair per standard; guarantee perfect reconstruction.
Common state space	**Net-new**	Standard-neutral economic state (stocks, flows, drivers) the simulation operates on. Retains a per-firm presentation map so native line items reproduce exactly. Specified as a knowledge graph over the standards' taxonomies (Section 3.2).
Accounting engine	**Reused**	Evolves stocks from flows with the accounting identities preserved by construction (no plugs, no circularity). Built by prior interns.
Driver-to-flow layer	**Net-new**	Maps a scenario to the flow variables with credible, non-linear relationships.

Table 1. System components. The extractor, the encode/decode adapters, the common state space, and the driver-to-flow layer are net-new; the accounting engine is reused from prior interns.

It is important that the encoder and decoder pass a round-trip test: encoding a structured statement into the state space and decoding it straight back, with no simulation in between, must reproduce that statement exactly. This tests the encode/decode pair alone, on already-structured input, and is separate from whether the filing was read correctly. Section 3.1 formalizes it.

3.1 Encoding, decoding, and perfect reconstruction

Why operate in a state space at all, rather than simulating the line items directly? First, line items are heterogeneous: firms and industries present the same economics under different labels, orderings, and levels of aggregation, so machinery written against native line items would fragment into per-firm and per-industry variants; the common state gives the simulation one stable schema. Second, standards become cheap to add: the driver-to-flow layer and the accounting engine are wired once to the standard-neutral state (the reused engine already operates on exactly this stock-and-flow representation), so a new standard costs an encode/decode adapter pair rather than a rewiring of drivers and accounting relations to a new set of standard-specific line items; the identities are also verified once, on one core, rather than per standard. Third, it is what makes Part II estimable: a conditional driver distribution is learned across many firms, and pooling firms requires that they share a representation. The price of this indirection is that nothing may be lost on the way in or out.

Formally, write s for a structured statement under a given standard, taken as already extracted from the filing (getting that extraction right is a separate problem, handled in Phase 1 with its own accuracy target). The encoder does not output only the economic state; it outputs a pair,

$$E(s) = (z,\, m), \tag{1}$$

where z is the standard-neutral economic state the simulation operates on (the stocks, flows, and drivers) and m is the per-firm presentation map: native line-item labels, their ordering and aggregation, sign conventions, and any presentation detail not captured by z. The decoder renders

a statement from that pair,

$$D(z,\, m) = \hat{s}. \tag{2}$$

Perfect reconstruction requires the round trip to return the original exactly, for every valid statement:

$$D\big(E(s)\big) = s, \qquad \text{equivalently} \qquad D \circ E = \mathrm{id}. \tag{3}$$

A decoder satisfying this (a *left inverse* of E) exists if and only if E is *injective*, meaning distinct statements never collapse to the same encoding:

$$s_1 \neq s_2 \implies E(s_1) \neq E(s_2). \tag{4}$$

If two different statements shared one encoding, the decoder would have to return both from the same input, which no function can do, so no lossless decoder could exist.

Making E injective, and so guaranteeing a left inverse, rests on two design facts. First, the encoding cannot fold independent information together: an injective map preserves dimension (a matrix is injective exactly when its rank equals its number of inputs, so it cannot land in fewer independent components), so z carries at least as many independent entries as the statement has independent line-item values. Two separate figures such as product and service revenue cannot be collapsed into one revenue number and later recovered, so z keeps them apart. The only entries the encoder may drop are those the accounting identities already fix, such as a subtotal determined by its parts or a total pinned by assets $=$ liabilities $+$ equity, which the decoder recomputes rather than stores; that reduction is lossless because the dropped values are exact functions of the kept ones. Second, z holds numbers but not presentation, so the map carries the remainder: line-item labels, ordering, sign conventions, and which subtotals are displayed. That is what separates two statements with identical numbers but different presentation, for instance the same figure labelled “Net sales” in one filing and “Total revenue” in another, giving $(z,\, m_1) \neq (z,\, m_2)$. Together these keep distinct statements at distinct encodings, since statements agreeing on both numbers and presentation are the same statement. The condition is a left inverse, not a full inverse: D need only be defined on encodings that came from real statements, not on every conceivable $(z,\, m)$.

The same map is what makes a forecast come out in the firm’s own statement. In a forecast the engine advances the state, $z \mapsto z'$, and the output is rendered through the retained map, $D(z',\, m)$, so the projected statement carries the firm’s native line items. Perfect reconstruction is the special case $z' = z$ (no simulation) that validates the encode-and-decode path; exact line-item parity in a real forecast is the same mechanism with z' in place of z.

3.2 Specifying the state space as a knowledge graph

Following the knowledge-graph direction the director has raised, the common state space is specified as a typed graph whose nodes are accounting concepts rather than line items, imported from the standard setters’ own machine-readable ontologies: the US GAAP and IFRS XBRL taxonomies (roughly 17,000 concepts each, maintained by FASB and the IFRS Foundation). Each node carries attributes the taxonomy already provides: period type (instant versus duration, which is precisely the stock versus flow distinction), balance polarity (debit or credit, the sign conventions), statement membership, and known label variants. Five edge types complete the graph: composition edges, imported from the calculation linkbase (subtotal arithmetic such as revenue equals product plus service); laws of motion, authored, recording which flows update which stocks and the cross-statement articulation (depreciation touching the income statement, the cash-flow add-back, and accumulated depreciation is one node in three places); label and synonym edges, imported; cross-standard correspondences, curated, with explicit non-correspondence where semantics diverge (LIFO has no IFRS counterpart); and driver attachments from Phase 3. The order

of magnitude is favorable: composition and labels are imported wholesale, and the authored layers cover only the simulated core, on the order of one to three hundred concepts, not seventeen thousand.

Encoding a statement then means building a firm overlay on this graph. Each native line resolves to a node: directly when tagged with a standard concept; as an anchored child of the nearest standard node when tagged with a firm extension; and, in the untagged PDF-only mode, through the firm's versioned mapping artifact: candidates proposed by an LLM at onboarding are accepted only if the induced composition subtree foots to the printed subtotals and the balance polarity matches the printed signs, then human-reviewed and frozen, and at run time resolution is an exact lookup in that artifact, with a line lacking an entry flagged under the abstain rule rather than resolved by a model (Section 5). This turns semantic mapping from a judgment call into a checkable, reviewable artifact. The firm's disclosed leaves become the entries of z, keyed by node; subtotals and totals are marked derived, dropped from z, and recomputed on decode (the one lossless reduction of Section 3.1); the presentation map m decorates the same overlay with exact native labels, display order, shown subtotals, signs, and scale. The engine walks the law-of-motion edges instead of a fixed vector, which resolves the variable-width concern of the Phase 1 scope note, and a leaf lacking its own driver inherits the nearest ancestor's dynamics, disaggregated by observed shares.

A worked example: a firm reports "Accrued warranty and related costs" under a company extension anchored to the standard warranty-accrual concept. The overlay adds it as a child of that node, from which it inherits instant period type (a stock), credit balance, membership in current liabilities (so it participates in the footing and A $=$ L $+$ E checks), a default law of motion (accrual$_t$ equals accrual$_{t-1}$ plus provisions minus claims), and the parent's driver linkage to product revenue, while m preserves the exact label so the forecast prints the firm's own words. In the untagged mode the same line is resolved by pruning label candidates against the constraint that it must foot into the printed current-liabilities total alongside its siblings.

Three cautions are built in: filed calculation linkbases are inconsistent across companies, so a firm's own arithmetic links are treated as soft constraints with a tolerance while the curated composition edges are the backbone; dimensional breakdowns are restricted to face-of-statement axes for the MVP; and taxonomy versions are pinned per fiscal year with explicit cross-version maps. One graph then serves five consumers: extraction disambiguation, the encode and decode adapters, the engine's stock-and-flow wiring, driver attachment, and cross-standard correspondence.

4. Phased build plan

Phases 0 through 3 build Part I, the MVP; Phase 4 is Part II, the learned-driver extension.

4.1 Phase 0: Acceptance harness and scope lock

Objective. Build the measuring stick before the system, since being wrong is not an option and the bar is accountant usability.

- Fix the firm set: roughly three US GAAP filers and three IFRS filers, with IFRS sourced from foreign private issuers that file Form 20-F (so the statements are iXBRL-tagged in EDGAR). Designate a smaller must-pass subset, for example two per standard, as the gating set.
- Assemble the test set of real annual reports with each prior-period statement hand-verified for ground truth.
- Fix the acceptance criteria with the director and an accountant: per-field extraction accuracy

and the definition of tool agreement; the perfect-reconstruction target; field-set parity and standard compliance for the output; and a directional scenario battery with the expected sign of each move.

Acceptance. The test set and written criteria are signed off, giving every later phase an objective pass or fail.

4.2 Phase 1: Extraction and state-space adapters (primary contribution)

Objective. Build two things: a robust extractor that reads a filing into a structured statement, and the per-standard encode/decode adapters with the common state space, so that extraction is accurate and the encode/decode round trip is lossless. The runtime extraction uses deterministic methods that cross-check each other; an LLM assists only at build time (Section 5). Sequence US GAAP first, then add IFRS as an extension on the same scaffolding.

Extraction (filing to structured statement), the robust part.

- Document paths, the core capability: a deterministic table parser on the HTML or PDF structure, and regular-expression and rule checks on the linearized text, each consuming a different representation of the filing; semantic mapping of native line items onto the graph is an exact lookup in a versioned per-firm mapping artifact, never a model call. These paths alone must meet a measured bar (the PDF-only mode of Section 5), since untagged documents are what generalize beyond public filings. Document scope for the untagged mode is born-digital PDFs: the statement pages must carry authored text (embedded fonts, no full-page raster with an invisible text overlay). Scanned or image-compiled documents, including OCR'd scans, are out of scope for the MVP; the runtime detects them mechanically and abstains rather than degrading silently. This removes an OCR dependency, not the redundancy argument: a corrupted text layer in a born-digital file still fails the layout and text readers together, which is what the identity checks and the remaining paths exist to catch.
- A build-time onboarding loop, the only place an LLM appears: for a new firm or a changed format, the model proposes the mapping artifact (native label to concept, sign, scale, structure); the knowledge graph validates the proposal mechanically (footing, polarity, identities, round trip); a human signs off; the artifact is versioned. At run time the system executes only reviewed code and frozen artifacts, and a line without an entry abstains and routes back to this loop.
- A tag path where the filing carries one: a deterministic reader of the filer's own iXBRL facts (SEC 10-K for US GAAP, Form 20-F for IFRS foreign private issuers, ESEF iXBRL as an alternative IFRS source), whose values carry explicit sign, scale, and period and never touch visual layout. The strongest and most decorrelated reader when present, and present for the MVP test set, but never a prerequisite.
- A reconciliation layer that accepts a field only when independent paths agree, flags disagreements for adjudication and human review with weighting by path independence (Section 5), runs accounting-identity checks (assets equal liabilities plus equity, subtotals foot, cash flow ties) with a rounding tolerance, and flags genuine inconsistencies or typos in the source filing.

Encode and decode (structured statement to and from the state space), the lossless part.

- Encode each standard's statements into the common state space, retaining a per-firm presentation map of native line items and labels.
- Decode the state back to native statements under each standard.

- Establish perfect reconstruction: encode then decode returns each structured statement exactly.

Scope note. The common state space carries the economic content shared across standards; recognition and measurement differences between US GAAP and IFRS (for example LIFO or development-cost capitalization) are handled inside each standard's own encode and decode. Cross-standard conversion (in under one standard, out under another) is out of scope: the round trip is always same-standard. Because the state carries whatever granularity each firm discloses, the state schema is effectively a union across firms, with each firm populating a different subset (a segment-reporting firm has product and service revenue components that a single-line firm lacks). This is lossless for reconstruction, but it means the Phase 3 driver layer must accommodate a variable-width state rather than a fixed vector, which is the natural place this design concentrates complexity. Section 3.2 sketches the mechanism: the engine and the driver layer walk the graph rather than a fixed vector, with drivers inherited down the concept hierarchy; the full proposal will specify it in detail.

Acceptance. Three gates. Extraction meets the per-field accuracy target on the test set for both standards, including a PDF-only ablation in which the tag path is withheld (measured at its own bar), with seeded read-errors caught and disagreements surfaced rather than silently resolved. The encode/decode pair achieves perfect reconstruction: the round trip through the state space reproduces each structured statement exactly under its own standard. And every run replays bit-identically from its logged lineage (input hash, code version, artifact version, pinned taxonomy version).

4.3 Phase 2: Engine integration and manual flow toggles

Objective. Connect a trusted baseline to the existing engine and let a user forecast by hand.

- Wire the encoded state from Phase 1 in as the initial state of the existing no-circularity engine.
- Expose the flow-variable parameters as manual toggles.
- Guarantee that every output preserves the input field set and balances by construction.

Acceptance. An end-to-end demo: a real filing is extracted and encoded, flows are toggled, and a balanced next-period statement is produced in the same field set under the same standard. This phase is mostly integration of existing work, so it is fast and gives an early end-to-end result.

4.4 Phase 3: Credible driver-to-flow relationships

Objective. Map scenarios to the flow variables with reasoning an accountant would accept, not a naive linear fit, and run the model forward stochastically.

- Use the existing external model (Jodie's) for the macro layer rather than rebuilding it.
- Build a simple state-space model for the industry and firm-specific layer, with reasoned, nonlinear scenario-to-flow relationships and a simple noise term, sufficient for an MVP.
- Connect the driver outputs to the flow inputs of the engine.
- Run simple stochastic simulation: for a scenario, sample the model's driver noise and propagate it through the engine (Monte Carlo) to a distribution of internally consistent statements.

Acceptance. The distribution passes the directional scenario battery from Phase 0 (it shifts the right way when the scenario changes), sampled statements survive the accountant "not obviously wrong" review on the test firms, and the spread is plausible. Most of the modeling judgment

lives here; the target is solid and robust, not fancy.

4.5 Phase 4: Learned driver distributions (Part II, conditional stretch)

Objective. Upgrade the driver distribution from simple noise to a learned model, for better-calibrated and correlated forecasts, once the MVP holds.

- Replace the simple driver noise with a learned conditional joint distribution of the drivers given firm state (the main modelling IP), capturing correlation across drivers.
- Estimate it from data with suitable structure (for example a factor structure on the residuals) and validate its calibration.

Note. Explicitly post-MVP and contingent on the earlier phases, consistent with deciding how much is achievable once results are in hand. Scheduled only in the final weeks and started only on MVP sign-off.

5. Robustness approach

Extraction, reading a filing into a structured statement, is held to a far higher precision bar than the rest of the system, because a misread figure invalidates everything downstream; the encode/decode step that follows is deterministic and lossless, so it carries no comparable error rate. The approach mirrors the engineering already used in the director's own tools: several readers cross-check one another, and a field is accepted only when they agree.

Under this gating rule the two failure modes have very different costs. If readers disagree, the field is flagged; that costs review time, not correctness. The expensive event is silent concordance, every reader returning the identical wrong value so the gate passes it. The 10^{-8} target therefore refers to the probability of concordant error, and for numeric fields such an error is demanding to produce by chance: independent readers must not merely both err, they must err onto the same wrong number. Three complementary filters defend it:

1. **Cross-reader agreement** catches discordant errors. With three to five readers, each below a 1% error rate and genuinely independent, the compound rate of an identical silent miss falls toward 10^{-8}; every disagreement is surfaced rather than silently resolved.
2. **Decorrelated input paths** are what the independence assumption actually requires, so each reader consumes a different representation of the same filing: the filer's own iXBRL tagged facts (explicit sign, scale, and period, with no dependence on visual layout); the HTML or PDF table structure via a deterministic layout parser; and the linearized text via regular-expression and rule checks; augmented by the prior year's filing, whose comparative column reports the same figures in a different document. Layout pathologies (a statement continuing onto a second page, negatives rendered as parentheses, a scale header such as "in thousands") fool layout-based readers the same way but do not touch the tag path; conversely, tag-path failures (custom extension tags, values reported only in footnotes) do not afflict the layout readers, so the intersection of failure modes shrinks.
3. **Accounting-identity checks** catch concordant errors that survive the first two filters, provided they break an identity: subtotals must foot, assets must equal liabilities plus equity, and the cash flow statement must tie, all within a rounding tolerance so ordinary rounding raises no false alarms. This filter compares the candidate extraction not to other readers but to constraints the true statement must satisfy, so even an error every reader agrees on is caught if it breaks the arithmetic. The same checks detect genuine typos in the source filing, which are surfaced to the user rather than silently patched, since the extractor's ground truth is what the

filing says.

LLMs at build time, determinism at run time. Production ingestion must be replicable and auditable to pass model approval: even an error must replay exactly. A hosted LLM in the inference path cannot meet that bar, for mundane numerical reasons: floating-point addition is not associative, and the summation order inside inference kernels depends on how the server batches concurrent traffic (tiling and reduction-split choices, and in mixture-of-experts models the routing and expert capacity that other users' tokens consume), so the low-order bits of the logits depend on load; greedy decoding turns a last-bit difference into a different token, and the autoregressive loop compounds it into a different answer.[1] Hosted endpoints also update silently, so there is no version to freeze or replay against. Accordingly, no model is consulted at run time. The LLM works only in change management: at onboarding it proposes a firm's mapping artifact, the knowledge graph validates the proposal mechanically, a human signs off, and the artifact is versioned. The runtime is pure reviewed code plus frozen artifacts and a pinned taxonomy, logging the input hash and the code and artifact versions, so any historical output, right or wrong, is exactly reproducible. When a filing's format drifts, the runtime abstains and flags rather than consulting a model, and the flag re-enters the onboarding loop. A trained driver model with frozen, versioned weights meets the same standard; the exclusion is of nondeterministic components in the inference path, not of statistical models.

Tags are the best case, not a prerequisite. Where a document carries no machine-readable tags (a private firm or another jurisdiction, provided the document is born-digital), the remaining readers still gate one another and the identity checks still bind, so reduced redundancy surfaces as a higher flag-for-review rate rather than as silent error. Because this untagged mode is the one that generalizes beyond tagged public filings, Phase 1 measures it separately through a PDF-only ablation in which the tag path is withheld and the accuracy is held to its own measured bar.

Residual risk. What remains silent is the intersection: errors concordant across decorrelated paths, identity-preserving, and within the rounding tolerance, for example a footnote-only value misread the same way by every text path and absent from the tags. Rare, but not zero, which is why the claim is "toward 10^{-8}, measured" rather than asserted. The Phase 1 seeded-error battery deliberately seeds the correlated pathologies above (continuation pages, parenthetical negatives, scale headers) to measure residual correlation rather than assume it away; it also seeds a scanned page and an OCR'd page into an otherwise born-digital document and requires the born-digital abstain to fire, so the scope limit is a tested behavior rather than a sentence. Disagreement adjudication is weighted by path independence: a standard-tag XBRL value outweighs two layout readers that agree with each other.

Why not a licensed dataset. Standardized products (Compustat and the like) reclassify each firm's native line items into a fixed common template; that is a non-injective encoding that discards the presentation the system must reproduce, so they cannot serve as the primary representation regardless of their accuracy. Vendor as-reported feeds preserve native lines but are themselves extraction pipelines with their own error rates, coverage, timeliness, and licensing constraints, a dependency the product cannot be built on. Most decisively, the capability under examination is ingestion of an arbitrary born-digital annual report, so outsourcing it would fail the test by construction. A licensed feed can still serve as an external reference reader in adjudication, with definitional

[1] Bit-exact greedy decoding is achievable self-hosted with batch-invariant kernels and a fully pinned stack, at a throughput cost (Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference," 2025). That path freezes an unexplainable component with a very large validation surface, and determinism buys replay, not explanation; the mapping artifact provides both and is small enough to review.

and restatement mismatches treated as flags to investigate rather than as extraction errors.

6. Open questions for alignment

The standards scope (US GAAP and IFRS), the precision asymmetry, and exact line-item parity are settled and folded into the plan above. The following remain open:

1. What does the external model (Jodie's) output, and at what granularity, so the state-space layer can consume it? To confirm with Jodie.
2. Who is the accountant reviewer, and how many statements across how many scenarios constitute a pass? A provisional definition is in place; the specific counts and reviewer are to be finalized after further discussion.
3. Which mode gates the Part I acceptance test: tagged public filings, or an untagged born-digital PDF (a private firm, another jurisdiction) at the same bar? The pipeline is built for both and the PDF-only mode is measured separately; the question is which bar the MVP must clear.

7. Timeline (internship window)

Phase	Focus	Window	Weeks
Phase 0	Scope lock and acceptance harness	31 Aug to 11 Sep 2026	2
Phase 1	Extraction and encode/decode adapters plus common state space (US GAAP first, then IFRS)	14 Sep to 13 Nov 2026	9
Phase 2	Engine integration and manual flow toggles	16 Nov to 27 Nov 2026	2
Phase 3	Driver-to-flow model, simple stochastic simulation, and scenario validation	30 Nov to 18 Dec 2026	3
Hardening	MVP hardening, accountant review, and buffer	21 Dec 2026 to 22 Jan 2027	$\sim$4
Phase 4	Learned driver distributions (conditional stretch)	25 Jan to 26 Feb 2027	$\sim$5

Table 2. Indicative timeline across the internship window (31 August 2026 to 26 February 2027). Phases 0 to 3 constitute Part I (the MVP); Phase 4 is Part II.

Key milestones. MVP feature-complete by 18 December 2026; accountant sign-off by 22 January 2027; Phase 4 begins only on sign-off.

The common-state-space and round-trip requirements enlarge Phase 1 (now nine weeks, US GAAP first and IFRS as an extension), which compresses Phase 3; the hardening band in late December absorbs any Phase 3 spillover. The schedule fits the internship window (31 August 2026 to 26 February 2027) and durations remain indicative, to be adjusted as scope is confirmed.

8. Next step

This is a rough draft intended to confirm direction. Once the open questions are settled, the full proposal will expand each phase with detailed methods, the exact accuracy metrics and test set, the state-space specification, the specific tool choices, and a finalized timeline.